ARCHITECTURE & TECHNICAL MANUAL

# TicketAI

## Smart Classification Engine Architecture Documentation

| | |
|---|---|
| TARGET RUNTIME | Cloudflare Workers (via Nitro) |
| LLM ENGINE | Groq (llama-3.3-70b-versatile) |
| REPOSITORY | github.com/AndileP3/create-preview-share |
| FRAMEWORK | TanStack Start v1 • React 19 • Vite 7 |
| DATE GENERATED | June 1, 2026 |

## 1. Executive Overview

**TicketAI** is a high-performance, single-page intelligent automation system engineered for swift processing and routing of internal organizational helpdesk items. Users interact with a refined, dark-mode single-page UI to input title, context descriptions, assignment priorities, and user identities.

Upon request submission, the system utilizes high-throughput inference nodes via the Groq Chat Completions interface running `llama-3.3-70b-versatile`. The underlying model acts as a highly deterministic categorization filter, processing natural language narratives to match requests seamlessly to their targeted institutional business unit: **HR**, **IT**, **Finance**, or **Operations**.

> **Architectural Paradigm**
>
> The application encapsulates a modular, pure-client operational prototype inside an enterprise-grade SSR shell wrapper (TanStack Start). This structure maintains a clean functional separation, making it trivial to ship to modern edge compute fabrics such as Cloudflare Workers.

## 2. System Integration & Core Features

- **Reactive Input Form:** Validates user metadata parameters including Title, Detailed Context Description, Priority Matrix, and Requester Profile identifiers.
- **Deterministic AI Router:** Interfaces via standard secure JSON schema requests directly onto `https://api.groq.com/openai/v1/chat/completions`. The engine supplies custom structured systems profiling prompts to guarantee response compliance within the four allowed destination channels.
- **Stateful Client Persistence:** Implements instant client-side rendering capabilities backed by synchronous browser storage mechanisms ( `localStorage` ) to maintain system history between context resets.
- **Live View Dashboard:** Provides full content filtering by target departments, dynamic regex-matched lexical searching over textual keys, and dynamic CSS styling enhancements.

## 3. Design System & Visual Grammar

The presentation shell honors a sophisticated developer-centric aesthetic engineered specifically to look sleek in production dashboard setups.

| Token Parameter | Color Value | Application Range |
| --- | --- | --- |
| **Primary Canvas Background** | `#0a0d14` | Global viewport layer and outermost viewport boundaries. |
| **Surface Elevations** | `#1c2438` | Card backgrounds, left fixed navigation columns, and inputs. |
| **Accent Blue Highlight** | `#3b82f6` | Focus outlines, interactive state triggers, and glow matrices. |
| **Department: HR** | `#EC4899` | Human Resource classification badges and routing links. |
| **Department: IT** | `#3B82F6` | Technical Operations and Infrastructure tags. |
| **Department: Finance** | `#10B981` | Fiscal tracking, invoicing, and ledger ticket routing. |
| **Department: Operations** | `#F59E0B` | Logistics, supply chain, and general workspace processing. |

## 4. Technical Stack Architecture

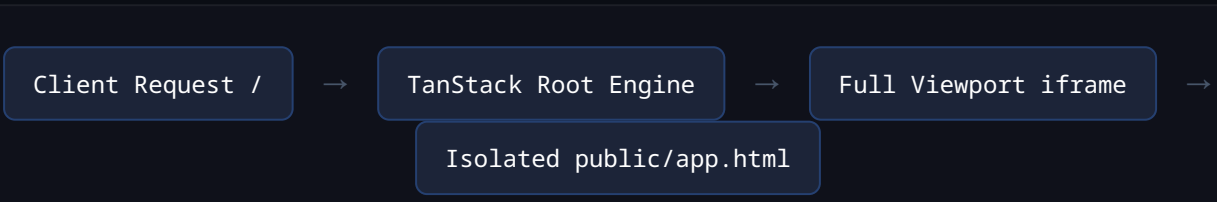
The workspace incorporates cutting-edge layout and processing tools optimized for ultra-low interaction delays and robust bundle sizes:

- **Framework Framework:** TanStack Start v1 layered cleanly above React 19 core features and Vite 7 asset compiling configurations.
- **Routing Strategy:** Fully file-system decoupled schemas managed automatically via standard structural tree schemas ( `@tanstack/react-router` yielding `routeTree.gen.ts` ).
- **Shell Package Manager:** Bun configurations utilizing high performance locked dependency mapping environments ( `bun.lock` and `bunfig.toml` ).
- **Target Deploy Target:** Cloudflare Workers system environments handled natively using target custom abstractions provided via Nitro engines and `@lovable.dev/vite-tanstack-config` .

## 5. Pipeline Orchestration & Structural Iframe Bridging

To combine the velocity of static client-driven design prototyping with robust edge hosting layers, the framework adopts a smart hybrid sandboxing technique. The core operational system layout (comprising 886 lines of reactive client runtime) is isolated within `public/app.html`.

The main TanStack active route at root `src/routes/index.tsx` acts as a seamless gateway. It spins up a viewport-locked, frame-borderless canvas that projects the standalone dashboard cleanly across all browser breakpoints.




> **Security & Key Access Notice**
>
> The standard environment variable `VITE_GROQ_API_KEY` is initialized out of local project root configurations. For direct standalone client capability execution within the preview tool, this string maps onto `GROQ_API_KEY` in the compiled file `public/app.html` at line ~640. Please utilize short-lived keys or configure reverse proxy routing blocks in restricted corporate networks.

## 6. Technical Repository Directory Structure

```
.
├── .env                          # Local Environment Configuration Key variables
├── .lovable/project.json         # Lovable template marker system indicator
├── bun.lock / bunfig.toml        # Bun fast performance structural package hashes
├── package.json                  # Runtime dependencies: React 19, Tailwind v4, Zod
├── vite.config.ts                # Cloudflare Worker Compilation Rules (via Nitro)
├── public/
│   └── app.html                  # ⭐ Consolidated Logic Engine (886 lines HTML/JS/CSS)
└── src/
    ├── server.ts                 # Base Server SSR initialization wrapper
    ├── styles.css                # Base Tailwind v4 Theme Tokens for application shell
    └── routes/
        ├── __root.tsx            # Global Layout context definition schema
        └── index.tsx             # ⭐ Full-viewport runtime portal layer (<iframe>)
```

## 7. Execution Lifespans & Core Workflows

1. The client initializes the connection sequence towards the root route endpoint `/`.
2. The internal runtime edge handlers construct the HTML wrapper document dynamically based on configurations in `__root.tsx`.
3. The secondary sandbox context renders the isolated HTML target tree from `public/app.html` inside a secure client iframe container.
4. When a new issue is submitted, the internal runtime directly posts to Groq's high-performance AI inference servers.
5. The client injects the returned payload asynchronously into the in-memory arrays and shifts UI badges dynamically without triggering full page reloads.